

Java Reghub Student Guide

1. What This App Does

Java Reghub is a small Spring Boot web app.

It lets a user:

- open a web page
- create a post with a title and an image
- see all posts on the page
- view each post image
- delete a post

The app is useful for learning:

- Spring Boot
- MVC layers
- database access with JPA
- file upload
- AWS S3 storage
- Jenkins CI/CD basics

2. Simple User Flow

This is what happens when a user creates a post:

1. The user opens 'http://localhost:8080'.
2. The browser shows the 'posts.html' page.
3. The user enters a title and chooses an image.
4. The browser sends the form to 'POST /posts'.
5. The controller receives the request.
6. The controller calls the service.
7. The service checks the title and image.
8. The service uploads the image to S3.
9. The service saves the post details in MySQL.
10. The app redirects back to the home page.
11. The new post is shown on the page.

3. Main Layers

The app has these main layers:

- View layer: shows HTML pages to the user
- Controller layer: receives browser requests
- Service layer: contains business logic
- Repository layer: talks to the database
- Model layer: describes database data
- Storage layer: talks to AWS S3

- Configuration layer: creates setup objects used by Spring

Each layer has a clear job. This makes the app easier to understand, test, and change.

4. View Layer

Main file:

- 'src/main/resources/templates/posts.html'

The view is the web page seen by the user.

It does three main things:

- shows a form to create a post
- shows success messages after create or delete
- shows all posts in a grid

The form sends data to:

- 'POST /posts'

Each image is loaded from:

- 'GET /posts/{id}/image'

Each delete button sends data to:

- 'POST /posts/{id}/delete'

The view uses Thymeleaf. Thymeleaf lets Java data appear inside HTML.

Example:

- the controller gives the page a list called 'posts'
- Thymeleaf loops through 'posts'
- each post becomes one card on the page

5. Controller Layer

Main file:

- 'src/main/java/com/reghub/controller/PostController.java'

The controller is like the front desk of the app.

It receives requests from the browser and decides what should happen next.

It does not do heavy work itself. It calls the service layer.

Home Page Request

Method:

- 'index'

URL:

- 'GET /'

What it does:

- asks the service for all posts
- puts the posts into the page model
- returns the 'posts' view

Simple meaning:

The browser asks for the home page. The controller gets the posts and shows the page.

Create Post Request

Method:

- 'createPost'

URL:

- 'POST /posts'

What it receives:

- 'title'
- 'image'

What it does:

- sends the title and image to 'PostService'
- redirects back to the home page

Simple meaning:

The browser sends a new post. The controller passes it to the service.

Image Request

Method:

- 'image'

URL:

- 'GET /posts/{id}/image'

What it does:

- asks the service to load the image for the post id
- sends image bytes back to the browser
- sets the image content type

Simple meaning:

The browser asks for an image. The controller returns the image data.

Delete Post Request

Method:

- 'deletePost'

URL:

- 'POST /posts/{id}/delete'

What it does:

- asks the service to delete the post
- redirects back to the home page

Simple meaning:

The browser asks to delete a post. The controller tells the service to delete it.

6. Service Layer

Main file:

- 'src/main/java/com/reghub/service/PostService.java'

The service layer contains the main app logic.

It is the brain of the app.

It decides the steps needed to complete each action.

The service uses:

- 'PostRepository' for database work
- 'StorageService' for image storage work

Find All Posts

Method:

- 'findAll'

What it does:

- asks the repository for all posts
- sorts newest posts first

Simple meaning:

Get the posts that should appear on the page.

Create a Post

Method:

- 'createPost'

What it does:

- checks that the title is not empty
- checks that an image was uploaded
- uploads the image using the storage service

- creates a 'Post' object
- saves the post using the repository

Simple meaning:

Save the image first, then save the post record in the database.

Load an Image

Method:

- 'loadImage'

What it does:

- finds the post by id
- gets the image key from the post
- loads the real image from S3

Simple meaning:

Use the database to find which image belongs to a post, then get that image from S3.

Delete a Post

Method:

- 'deletePost'

What it does:

- finds the post by id
- deletes the post from the database
- tries to delete the image from S3
- writes a warning log if the S3 delete fails

Simple meaning:

Remove the database record and also try to remove the uploaded image.

7. Repository Layer

Main file:

- 'src/main/java/com/reghub/repository/PostRepository.java'

The repository layer talks to the database.

This app uses Spring Data JPA.

The repository extends:

- 'JpaRepository<Post, Long>'

This gives the app many database methods automatically, such as:

- save
- find by id

- delete
- find all

The app also defines one custom method:

- 'findAllOrderByCreatedAtDesc'

Simple meaning:

Get all posts from the database, newest first.

8. Model Layer

Main file:

- 'src/main/java/com/reghub/model/Post.java'

The model describes the data saved in the database.

The 'Post' class maps to the 'posts' table.

Each post has:

- 'id': unique database id
- 'title': post title
- 'imageUrl': public URL for the uploaded image
- 'imageKey': S3 key used to find the image
- 'createdAt': time when the post was created

The '@Entity' annotation tells JPA:

- this class should be stored in the database

The '@Table(name = "posts")' annotation tells JPA:

- use the 'posts' table

The '@PrePersist' method sets 'createdAt' before the post is first saved.

Simple meaning:

The model is the Java version of a database row.

9. Storage Layer

Main files:

- 'StorageService.java'
- 'S3StorageService.java'
- 'StoredFile.java'
- 'StoredObject.java'
- 'StorageException.java'

The storage layer handles image files.

The app stores images in AWS S3.

StorageService

This is an interface.

It says what storage must be able to do:

- store a file
- load a file
- delete a file

Simple meaning:

It is a contract for file storage.

S3StorageService

This is the real storage class.

It uses AWS S3.

When storing an image, it:

- reads the uploaded file
- creates a unique file key
- uploads the file to S3
- returns the S3 key and public URL

When loading an image, it:

- gets the image bytes from S3
- returns the image content and content type

When deleting an image, it:

- deletes the object from S3

Simple meaning:

This class is responsible for talking to AWS S3.

10. Configuration Layer

Main file:

- 'src/main/java/com/reghub/config/S3Config.java'

This class creates the AWS S3 client.

Spring uses the '@Bean' method to create an 'S3Client'.

The region comes from:

- 'app.aws.region'

Simple meaning:

This layer prepares tools that other classes need.

11. Application Start

Main file:

- 'src/main/java/com/reghub/ReghubApplication.java'

This is the start of the Spring Boot app.

The 'main' method runs:

- 'SpringApplication.run(...)'

Simple meaning:

This starts the web server and loads all Spring components.

12. Database

The app uses MySQL.

The connection settings are in:

- 'src/main/resources/application.properties'

Important settings:

- database URL
- database username
- database password
- JPA table update setting

The app uses:

- 'spring.jpa.hibernate.ddl-auto=update'

Simple meaning:

Hibernate can update the database table structure when the app starts.

13. Environment Variables

The app can read values from environment variables.

Examples:

- 'SPRING_DATASOURCE_URL'
- 'SPRING_DATASOURCE_USERNAME'
- 'SPRING_DATASOURCE_PASSWORD'
- 'AWS_REGION'
- 'AWS_S3_BUCKET'
- 'AWS_ACCESS_KEY_ID'
- 'AWS_SECRET_ACCESS_KEY'

Simple meaning:

Secrets and environment-specific values should not be hard-coded in Java code.

14. Logs

The app uses normal Spring Boot logging.

Right now, logs go to the console by default.

That means:

- if you run the app in a terminal, logs appear in the terminal
- if you run the app in Docker, logs appear in container logs
- no special log file is configured in the code

To save logs to a local file, run:

```
java -jar target/java-reghub-0.0.1-SNAPSHOT.jar > app.log 2>&1
```

15. Full Request Flow

Create post flow:

Browser

- > PostController
- > PostService
- > S3StorageService
- > AWS S3
- > PostRepository
- > MySQL database
- > back to Browser

Load home page flow:

Browser

- > PostController
- > PostService
- > PostRepository
- > MySQL database
- > posts.html
- > Browser

Load image flow:

Browser

- > PostController
- > PostService
- > PostRepository
- > MySQL database
- > S3StorageService
- > AWS S3
- > Browser

Delete post flow:

Browser

- > PostController
- > PostService
- > PostRepository
- > MySQL database
- > S3StorageService
- > AWS S3
- > Browser

16. Easy Way To Remember

Use this simple sentence:

The controller receives the request, the service decides what to do, the repository saves database data, and the storage service handles image files.

17. Good Student Questions

Ask students:

- Why should the controller not contain all the logic?
- Why do we use a service layer?
- What does the repository do for us?
- Why store the image in S3 but post details in MySQL?
- What would happen if the image upload fails?
- What would happen if the database save fails?
- Where do logs appear when the app runs?

18. Summary

Java Reghub is a simple post application.

It teaches how a Spring Boot app is organized into layers.

Each layer has one main responsibility:

- View: shows the page
- Controller: receives browser requests
- Service: contains business rules
- Repository: talks to the database
- Model: represents database data
- Storage: handles uploaded images
- Config: creates shared setup objects

This structure helps students understand how real Java web applications are built.